# USACO Gold: Digit DP

Mmukul Khedekar

2 February 2026

## Contents

**1 Focus Problem: Odometer** **2**

**2 General Resources** **3**
2.1 DP-book Chapter 5 . . . 3
2.2 Geeksforgeeks Digit DP . . . 5
2.3 Introduction to Digit DP by Kartik Arora . . . 5
2.4 Digit DP Codeforces Blog . . . 5

**3 Problems** **5**
3.1 Codeforces 100886G . . . 5
3.2 SPOJ Digit Sum . . . 5
3.3 Codeforces 628D . . . 5

**4 USACO** **5**
4.1 USACO 2023 Feb, Gold — Problem 3, Piling Papers . . . 5
4.2 USACO 2021 Feb, Gold — Problem 3, Count the Cows . . . 5

# §1 Focus Problem: Odometer

**Problem 1.1** (USACO 2014 US Open (Silver) — Problem 3, Odometer)

Farmer John's cows are on a road trip! The odometer on their car displays an integer mileage value, starting at $X$ ($100 \leq X \leq 10^{18}$) miles at the beginning of the trip and ending at $Y$ ($X \leq Y \leq 10^{18}$) miles at the end.

Whenever the odometer displays an *interesting* number (including at the start and end of the trip), the cows will moo.

A number is called *interesting* if, when considering all its digits (excluding leading zeros), at least half of the digits are the same. For example, the numbers 3223 and 110 are interesting, while 97791 and 123 are not.

Your task is to determine how many times the cows will moo during the trip.

### Input

The first line contains two integers $X$ and $Y$.

### Output

Output a single integer: the number of times the cows moo.

### Sample Input

```
110 133
```

### Sample Output

```
14
```

### Explanation

The cows moo when the odometer reads:

$$110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 121, 122, 131, 133.$$

Let's start with a brute force approach. The straightforward brute force is to iterate over all numbers in the range $X$ to $Y$ and for each number check if it is *interesting*. To check if a number is interesting, we just iterate over its digits and maintain a frequency array. The total time complexity should be

$$\mathcal{O}\left((Y - X)\log Y\right)$$

# §2 General Resources

## §2.1 DP-book Chapter 5

The important thing is motivating the dp states for digit dp. From what I could understand, if we scan a number from left to right then we can compute the required property for the current number prefix using the already computed property for the number prefixes. For example, let's do the following problem.

**Problem 2.1**

There are two integers $n$, $m$ where $1 \leq n \leq m \leq 10^{18}$. Find the number of integers between $n$ and $m$ inclusive such that contain exactly $k$ digits $d$ where $1 \leq d \leq 9$ and $k \geq 0$.

The main idea is that if we know the count of numbers such that on a prefix the digit $d$ appears $j$ times, then

$$dp[i][j][d] \xleftarrow{d'=d} dp[i-1][j-1][d]$$
$$dp[i][j][d] \xleftarrow{d' \neq d} dp[i-1][j][d]$$

So the three states we need to keep track of are

- $i$, the length of the prefix of the number we have scanned
- $j$, the frequency of the digit $d$
- (above), if the number is greater than the target number. This last state could be 0, 1 or 2 depending upon
  - 0, count of numbers strictly smaller than the target value.
  - 1, count of numbers exactly equal to the target value.
  - 2, count of numbers greater than the target value.

  Therefore, if
  - (above) is 0, adding any number will make the prefix still smaller that the target number $\implies$ (above) remains 0.
  - (above) is 1, adding a number can make the prefix either smaller, equal or greater. This can be computed using

    $$(d' \geq i^{\text{th}} \text{ digit}) + (d' > i^{\text{th}} \text{ digit})$$

  - (above) is 2, adding any number will make the prefix still greater than the target number $\implies$ (above) remains 2.

We don't need to worry about leading zeroes in this problem, otherwise we will need another boolean state to keep track if the number has leading zeroes or not. For the base case, initialise the entire dp table by 0. The final answer is computed by

$$\boxed{dp[0][k][0] + dp[0][k][1]}$$

## Code

```
Answer checker(i64 m, i64 k, i64 d) {
    i64 dp[20][20][3];
    std::fill(&dp[0][0][0], &dp[0][0][0] + 20*20*3, 0LL);

    int digit = (m / pow10[18]) % 10;
    for (int D = 0; D <= 9; D++) {
        dp[18][(d == D)][(D >= digit) + (D > digit)] += 1;
    }

    for (int i = 18; i > 0; i--) {
        int digit = (m / pow10[i - 1]) % 10;
        for (int j = 0; j <= 19; j++) {
            for (int D = 0; D <= 9; D++) {
                dp[i - 1][j + (d == D)][(D >= digit) + (D > digit)]
                    += dp[i][j][1];

                dp[i - 1][j + (d == D)][0]
                    += dp[i][j][0];

                dp[i - 1][j + (d == D)][2]
                    += dp[i][j][2];
            }
        }
    }

    return { dp[0][k][0] + dp[0][k][1] };
}

void solve() {
    i64 n, m, k, d;
    std::cin >> n >> m >> k >> d;

    i64 answer = checker(m, k, d).answer;
    if (n > 0) answer -= checker(n - 1, k, d).answer;

    std::cout << answer << '\n';
}
```

Now we will solve a slightly harder variant of this problem. We need to account for leading zeroes for this one.

### Problem 2.2

There are two integers $n$, $m$ where $0 \leq n \leq m \leq 10^{18}$. Find the number of integers between $n$ and $m$ inclusive such that contain exactly $k$ digits $d$ where $0 \leq d \leq 9$ and $k \geq 0$.

### §2.2 Geeksforgeeks Digit DP

### §2.3 Introduction to Digit DP by Kartik Arora

### §2.4 Digit DP Codeforces Blog

## §3 Problems

### §3.1 Codeforces 100886G

### §3.2 SPOJ Digit Sum

### §3.3 Codeforces 628D

## §4 USACO

### §4.1 USACO 2023 Feb, Gold — Problem 3, Piling Papers

### §4.2 USACO 2021 Feb, Gold — Problem 3, Count the Cows